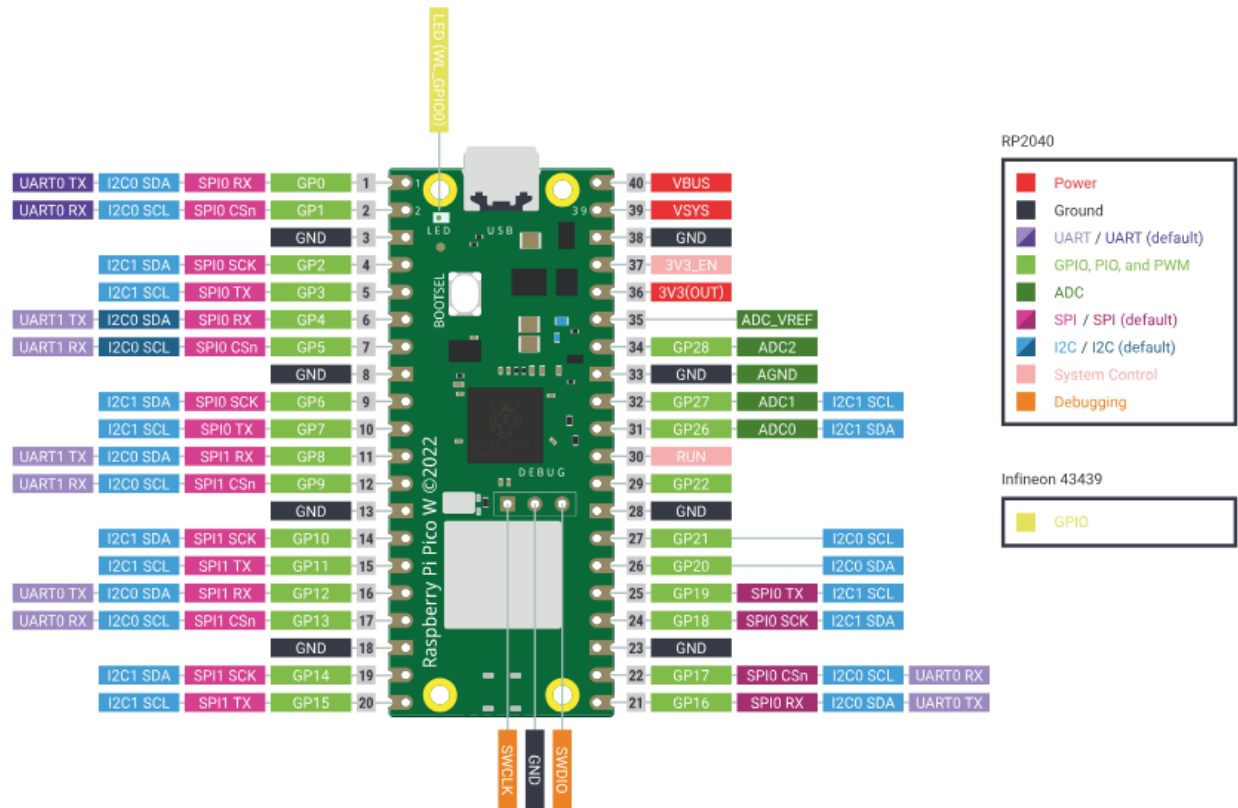


USER-SELECTED LED BLINK USING BITWISE OPERATORS” ON THE RASPBERRY PI PICO-W USING C SDK



GUIDED BY:
DURAI P S

MENTOR:
DANIEL RAJ A

TEAM:

1. KARTHICK A
2. LUCKSHANA V R
3. SHREE SUBI G
4. JANASRUTHIKA P R

TABLE OF CONTENT

S.no	Content	Page number
1.	<u>Cloning sdk and examples to the directory</u>	<u>3</u>
2.	<u>Create project folder & export SDK path</u>	<u>4</u>
3.	<u>Build the project using CMake and Make</u>	<u>6</u>
4.	<u>Copy .uf2 file to flash the microcontroller</u>	<u>7</u>
5.	<u>Appendix</u>	<u>8</u>

I. **CLONING SDK AND EXAMPLES TO THE DIRECTORY:**

1.1. To clone the SDK, Create a new directory and clone the SDK as given below

```
$ mkdir pico
$ cd pico
$ git clone -b master https://github.com/raspberrypi/pico-sdk.git
$ cd pico-sdk
$ git submodule update --init
```

1.2. To build applications we need CMake, ARM GNU Toolchain. To install those, run the command.

```
$ sudo apt update
$ sudo apt install cmake gcc-arm-none-eabi build-essential libnewlib-
arm-none-eabi minicom
```

- **Cmake** = CMake is an open-source, cross-platform tool used for managing the build process of software projects. It allows developers to define the build process in a platform-independent way by creating configuration files (CMakeLists.txt)
- **Gcc-arm-none-eabi** = Compiling C/C++ code for ARM Cortex-M microcontrollers.
Bare-metal development, meaning there's no OS — you directly program the hardware.
- **libnewlib-arm-none-eabi** = A C standard library (like stdio.h, math.h) optimized for embedded systems.
- **build-essential** = A meta-package that installs tools like make, g++ and other basic development tools needed to compile C/C++ code.

II. CREATE PROJECT FOLDER AND EXPORT SDK PATH:

2.1. When doing a new project, create a project folder:

```
$ cd ..  
$ mkdir led_input_project  
$ cd led_input_project
```

2.2. Create main.c :

```
nano main.c
```

```
#include <stdio.h>  
#include "pico/stdlib.h"  
  
#define NUM_LEDS 8  
const uint LED_PINS[NUM_LEDS] = {0, 1, 2, 3, 4, 5, 6, 7};  
  
void set_led(uint8_t led_num) {  
    for (int i = 0; i < NUM_LEDS; i++) {  
        gpio_put(LED_PINS[i], 0); // Clear all LEDs  
    }  
  
    if (led_num >= 1 && led_num <= NUM_LEDS) {  
        uint8_t bitmask = 1 << (led_num - 1);  
        for (int i = 0; i < NUM_LEDS; i++) {  
            if (bitmask & (1 << i)) {  
                gpio_put(LED_PINS[i], 1);  
            }  
        }  
        printf("LED %d is ON\n", led_num);  
    } else {  
        printf("Invalid LED number. Enter 1 to 8.\n");  
    }  
}  
  
int main() {
```

```

    stdio_init_all();

    while (!stdio_usb_connected()) {
        sleep_ms(100);
    }

    for (int i = 0; i < NUM_LEDS; i++) {
        gpio_init(LED_PINS[i]);
        gpio_set_dir(LED_PINS[i], GPIO_OUT);
    }

    printf("Enter LED number (1-8):\n");

    while (1) {
        int ch = getchar_timeout_us(1000000);
        if (ch != PICO_ERROR_TIMEOUT && ch >= '1' && ch <= '8') {
            set_led(ch - '0');
        } else if (ch != PICO_ERROR_TIMEOUT) {
            printf("Invalid input.\n");
        }
    }
}

```

2.3. Create CMakeLists.txt :

```
nano CMakeLists.txt
```

```

cmake_minimum_required(VERSION 3.13)
include(pico_sdk_import.cmake)

project(led_input_project C CXX ASM)
pico_sdk_init()

add_executable(led_input main.c)

# Enable USB output (for printf, getchar, etc.)

```

```

pico_enable_stdio_usb(led_input 1)
# Disable UART (optional — use 0 to disable, or 1 to keep it)
pico_enable_stdio_uart(led_input 0)

# Link standard Pico SDK libraries
target_link_libraries(led_input pico_stdlib)

# Generate UF2, bin, and hex files
pico_add_extra_outputs(led_input)

```

2.4. Create `pico_sdk_import.cmake`:

```
$ cp ~/pico/pico-sdk/external/pico_sdk_import.cmake .
```

III. BUILD THE PROJECT USING CMAKE AND MAKE:

3.1. Compile the program.

```

$ cd ~/pico/led_input_project
$ mkdir build
$ cd build
$ cmake ..
$ make

```

```

[ 90%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/class/net/ecn_rndis_device.c.o
[ 91%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/class/net/ncm_device.c.o
[ 93%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/class/usb/lsm/lsm_device.c.o
[ 94%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/class/vendor/vendor_device.c.o
[ 95%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/class/video/video_device.c.o
[ 96%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/tusb.c.o
[ 97%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/lib/tinyusb/src/common/tusb_fifo.c.o
[ 98%] Building C object CMakeFiles/led_input.dir/home/luckshana/pico/pico-sdk/src/rp2_common/pico_ftx/rp2040_usb_device_enumeration/rp2040_usb_device_enumeration.c.o
[100%] Linking CXX executable led_input.elf
[100%] Built target led_input
luckshana@luckshana-Inspiron-15-3511:~/lucky/pico/led_input_project/build$ ls
CMakeCache.txt      generated          led_input.elf     led_input.uf2     pico-sdk
CMakeFiles          led_input.bin     led_input.elf.map Makefile          pioasm
cmake_install.cmake led_input.dis     led_input.hex     pico_flash_region.ld pioasm-install
luckshana@luckshana-Inspiron-15-3511:~/lucky/pico/led_input_project/build$ cp led_input.uf2 /media/luckshana/RPI-RP2/
luckshana@luckshana-Inspiron-15-3511:~/lucky/pico/led_input_project/build$ ls /dev/ttyACM0
/dev/ttyACM0
luckshana@luckshana-Inspiron-15-3511:~/lucky/pico/led_input_project/build$ minicom -b 115200 -o -D /dev/ttyACM0

Welcome to minicom 2.8

OPTIONS: I18n
Port /dev/ttyACM0, 09:58:07

```

- If cmake file shows any error, update the CMake version after checking the version using.

```
cmake --version
```

```
$ sudo apt remove cmake  
$ sudo snap install cmake --classic
```

IV. COPY .UF2 FILE TO FLASH THE MICROCONTROLLER:

4.1. After the completion of progress, copy the uf2 file.

```
$ cp led_input.uf2 /media/$USER/RPI-RP2/
```

4.3. Connect with Minicom:

- List serial devices and launch minicom.

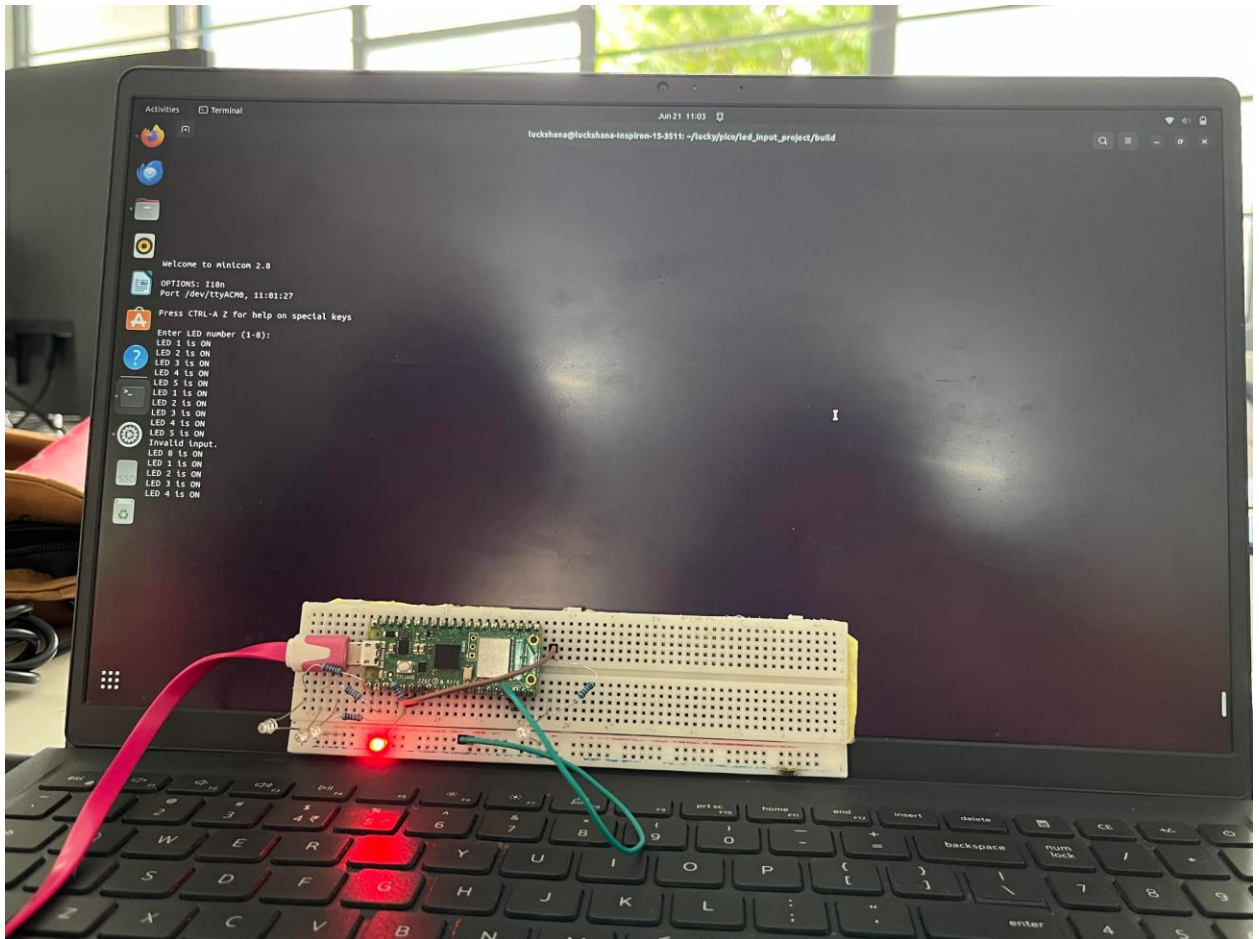
```
$ ls /dev/ttyACM*  
$ minicom -b 115200 -o -D /dev/ttyACM0
```

Note: To **exit Minicom**, press:

```
Ctrl + A, then press X → confirm exit.
```

4.4. Test Input:

- Type any number from 1 to 8..
- The corresponding LED turns ON.
- You'll see **LED x is ON** on screen.



VII.APPENDIX:

REFERENCE LIST:

1. <https://www.digikey.in/en/maker/projects/raspberry-pi-pico-and-rp2040-cc-part-1-blink-and-vs-code/7102fb8bca95452e9df6150f39ae8422>
2. <https://youtu.be/JhajoAyP8e4?si=tR607pWw90qvuItR>
3. https://wellys.com/posts/rp2040_c_blink/